



Estácio

1º Procedimento | Criação das Entidades e Sistema de Persistência

Nome da disciplina: Programação Back- end com Java

Campus: POLO ELDORADO III - CONTAGEM – MG

Curso: DESENVOLVIMENTO FULL STACK

Turma: DGT2821 – 9001

2026/1

Guilherme Rogério Souza Comunian

[Link do Repositório](#)

Sumário

Objetivo Geral	3
1 - Criar um projeto do tipo Ant..Java Application no NetBeans, utilizando o nome CadastroPOO para o projeto.	3
2 - Criar um pacote com o nome "model", para as entidades e gerenciadores.	3
3 - No pacote model criar as entidades, com as seguintes características.....	3
4 - No pacote model criar os gerenciadores, com as seguintes características.	6
5 - Alterar o método main da classe principal para testar os repositórios.....	9
6 - Ajustar as características para obter uma execução como a seguinte.....	10
7 - Verificar as funcionalidades solicitadas e os arquivos gerados.	10
Análise e Conclusão	10
a) Quais as vantagens e desvantagens do uso de herança?	10
b) Por que a interface Serializable é necessária ao efetuar persistência em arquivos binários?	10
c) Como o paradigma funcional é utilizado pela API stream no Java?	11
d) Quando trabalhamos com Java, qual padrão de desenvolvimento é adotado na persistência de dados em arquivos?.....	11

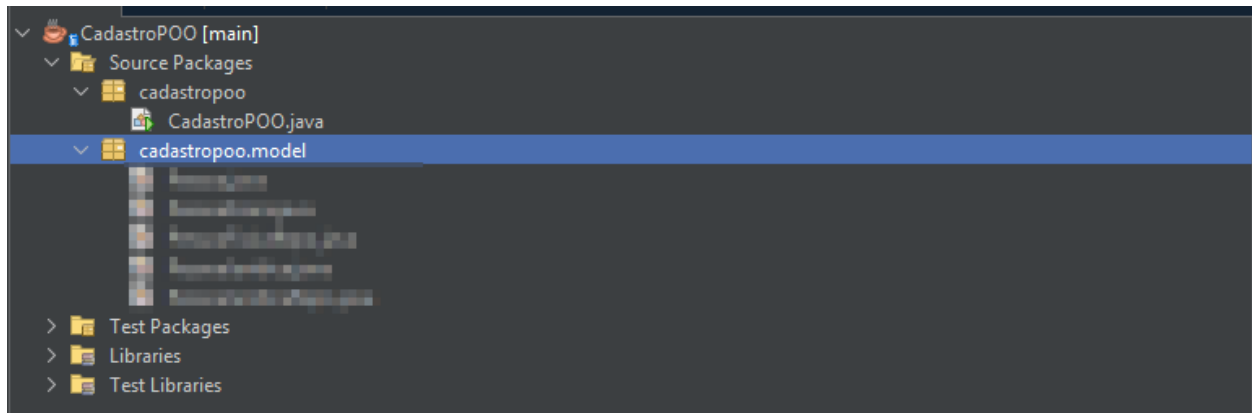
Objetivo Geral

Desenvolver um sistema em Java utilizando o paradigma de Programação Orientada a Objetos (POO), aplicando conceitos como herança, encapsulamento e polimorfismo, além de implementar mecanismos de persistência de dados em arquivos binários por meio da serialização de objetos, garantindo o armazenamento e recuperação das informações.

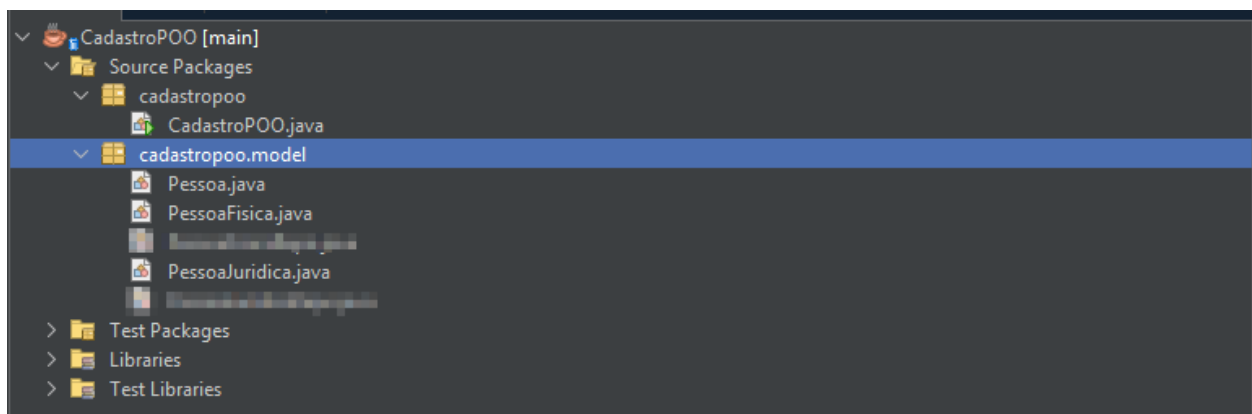
1 - Criar um projeto do tipo Ant..Java Application no NetBeans, utilizando o nome CadastroPOO para o projeto.



2 - Criar um pacote com o nome "model", para as entidades e gerenciadores.



3 - No pacote model criar as entidades, com as seguintes características.



Pessoa.java

```
package cadastrapoo.model;
import java.io.Serializable;

public class Pessoa implements Serializable {
    private int id;
    private String name;

    public Pessoa(int id, String name) {
        this.id = id;
        this.name = name;
    }

    public void show() {
        System.out.printf("Id: %d %nName: %s%n", id, name);
    }

    public int getId() { return this.id; }
    public void setId(int newId) { this.id = newId; }
    public String getName() { return this.name; }
    public void setName(String newName) { this.name = newName; }
}
```

PessoaFisica.java

```
package cadastrapoo.model;
import java.io.Serializable;

public class PessoaFisica extends Pessoa implements Serializable {
    private String cpf;
    private int age;

    public PessoaFisica(int id, String name, String cpf, int age) {
        super(id, name);
        this.cpf = cpf;
        this.age = age;
    }

    @Override
    public void show() {
        System.out.printf("Pessoa Fisica: %nId: %d %nName: %s%nCpf: %s %nAge: %d%n", getId(), getName(), this.cpf, this.age);
    }

    public int getAge() { return this.age; }
    public void setAge(int newAge) { this.age = newAge; }
    public String getCpf() { return this.cpf; }
    public void setCpf(String newCpf) { this.cpf = newCpf; }
}
```

PessoaJuridica.java

```
package cadastrapoo.model;
import java.io.Serializable;

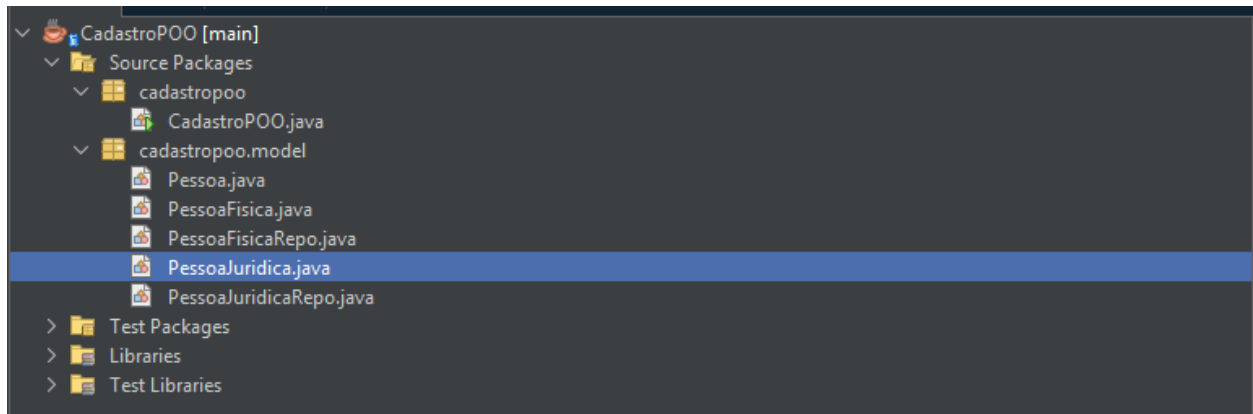
public class PessoaJuridica extends Pessoa implements Serializable {
    private String CNPJ;

    public PessoaJuridica(int id, String name, String CNPJ) {
        super(id, name);
        this.CNPJ = CNPJ;
    }

    @Override
    public void show() {
        System.out.printf("Pessoa Juridica: %nId: %d %nName: %s%nCNPJ: %s%n", getId(), getName(), this.CNPJ);
    }

    public String getCNPJ() { return this.CNPJ; }
    public void setCNPJ(String newCNPJ) { this.CNPJ = newCNPJ; }
}
```

4 - No pacote model criar os gerenciadores, com as seguintes características.



PessoaFisicaRepo.java

```
package cadastrpoo.model;
import java.io.*;
import java.util.ArrayList;

public class PessoaFisicaRepo {
    private ArrayList<PessoaFisica> list = new ArrayList<>();

    public void insert(PessoaFisica pf) {
        list.add(pf);
    }

    public void alter(PessoaFisica pf) {
        for (int i = 0; i < list.size(); i++) {
            if (list.get(i).getId() == pf.getId()) {
                list.set(i, pf);
            }
        }
    }

    public void delete(int id) {
        list.removeIf(p -> p.getId() == id);
    }

    public PessoaFisica get(int id) {
        for (PessoaFisica p : list) {
            if (p.getId() == id) return p;
        }
        return null;
    }

    public ArrayList<PessoaFisica> getAll() {
        return list;
    }

    public void keep(String arquivo) throws Exception {
        ObjectOutputStream out = new ObjectOutputStream(new FileOutputStream(arquivo));
        out.writeObject(list);
        out.close();
    }

    public void retrieve(String arquivo) throws Exception {
        ObjectInputStream in = new ObjectInputStream(new FileInputStream(arquivo));
        list = (ArrayList<PessoaFisica>) in.readObject();
        in.close();
        System.out.printf("Dados de Pessoa Fisica Recuperados%n");
    }
}
```

PessoaJuridicaRepo.java

```
package cadastrapoo.model;
import java.io.*;
import java.util.ArrayList;
import java.util.Locale;

public class PessoaJuridicaRepo {
    private ArrayList<PessoaJuridica> list = new ArrayList<>();

    public void insert(PessoaJuridica pj) {
        list.add(pj);
    }

    public void alter(PessoaJuridica pj) {
        for (int i = 0; i < list.size(); i++) {
            if (list.get(i).getId() == pj.getId()) {
                list.set(i, pj);
            }
        }
    }

    public void delete(int id) {
        list.removeIf(p -> p.getId() == id);
    }

    public PessoaJuridica get(int id) {
        for (PessoaJuridica p : list) {
            if (p.getId() == id) return p;
        }
        return null;
    }

    public ArrayList<PessoaJuridica> getAll() {
        return list;
    }

    public void keep(String arquivo) throws Exception {
        ObjectOutputStream out = new ObjectOutputStream(new FileOutputStream(arquivo));
        out.writeObject(list);
        out.close();
    }

    public void retrieve(String arquivo) throws Exception {
        ObjectInputStream in = new ObjectInputStream(new FileInputStream(arquivo));
        list = (ArrayList<PessoaJuridica>) in.readObject();
        in.close();
        System.out.printf("Dados de Pessoa Juridica Recuperados%n");
    }
}
```


5 - Alterar o método main da classe principal para testar os repositórios.

CadastroPOO.java

```
package cadastrapoo;
import cadastrapoo.model.*;
import java.util.Scanner;

public class CadastroPOO {

    public static void main(String[] args) {
        try {

            // =====
            //           First Part
            // =====

            PessoaFisicaRepo repo1 = new PessoaFisicaRepo();

            repo1.insert(new PessoaFisica(1, "Ana", "11111111111", 25));
            repo1.insert(new PessoaFisica(2, "Carlos", "22222222222", 52));
            System.out.printf("Dados de Pessoa Fisica Armazenados%n");

            repo1.keep("pf.dat");

            PessoaFisicaRepo repo2 = new PessoaFisicaRepo();
            repo2.retrive("pf.dat");

            for (PessoaFisica p : repo2.getAll()) {
                p.show();
            }

            PessoaJuridicaRepo repo3 = new PessoaJuridicaRepo();

            repo3.insert(new PessoaJuridica(3, "XPTO Sales", "33333333333"));
            repo3.insert(new PessoaJuridica(4, "XPTO Solutions", "44444444444"));
            System.out.printf("Dados de Pessoa Juridica Armazenados%n");

            repo3.keep("pj.dat");

            PessoaJuridicaRepo repo4 = new PessoaJuridicaRepo();
            repo4.retrive("pj.dat");

            for (PessoaJuridica p : repo4.getAll()) {
                p.show();
            }

        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

6 - Ajustar as características para obter uma execução como a seguinte.

```
run:
Dados de Pessoa Fisica Armazenados
Dados de Pessoa Fisica Recuperados
Pessoa Fisica:
Id: 1
Name: Ana
Cpf: 11111111111
Age: 25
Pessoa Fisica:
Id: 2
Name: Carlos
Cpf: 22222222222
Age: 52
Dados de Pessoa Juridica Armazenados
Dados de Pessoa Juridica Recuperados
Pessoa Juridica:
Id: 3
Name: XPTO Sales
CNPJ: 33333333333
Pessoa Juridica:
Id: 4
Name: XPTO Solutions
CNPJ: 44444444444
BUILD SUCCESSFUL (total time: 0 seconds)
```

7 - Verificar as funcionalidades solicitadas e os arquivos gerados.

As funções foram verificadas e funcionam conforme o solicitado. Além disso, os arquivos estão sendo gerados corretamente.

Análise e Conclusão

a) Quais as vantagens e desvantagens do uso de herança?

A herança possibilita a reutilização de código e a organização hierárquica das classes, evitando redundâncias e facilitando a manutenção. Como desvantagem, pode aumentar a complexidade do sistema, especialmente em projetos grandes, tornando o código mais difícil de entender e manter devido ao alto nível de abstração e acoplamento entre classes.

b) Por que a interface Serializable é necessária ao efetuar persistência em arquivos binários?

A interface Serializable é necessária para permitir que objetos sejam convertidos em uma sequência de bytes, possibilitando sua gravação em arquivos binários ou transmissão. Sem essa interface, o Java não permite que objetos sejam serializados. Ela não está relacionada à compactação, mas sim à transformação do objeto em formato persistente.

c) Como o paradigma funcional é utilizado pela API stream no Java?

O paradigma funcional é utilizado na API Stream do Java por meio do uso de expressões lambda e operações encadeadas, permitindo manipulação de coleções de forma declarativa. Com isso, é possível aplicar operações como filtragem, transformação e agregação de dados sem modificar o estado original dos objetos, tornando o código mais conciso e legível.

d) Quando trabalhamos com Java, qual padrão de desenvolvimento é adotado na persistência de dados em arquivos?

O padrão de desenvolvimento adotado para persistência em arquivos em Java é baseado no uso de serialização de objetos e no padrão de repositório (Repository), onde classes específicas são responsáveis por gerenciar o armazenamento, recuperação e manipulação dos dados, separando a lógica de persistência da lógica de negócio.